SOFTWARE TEST PLAN

Online Quiz Management System

Project Name: Online Quiz Management System

---

1. Introduction

The Online Quiz Management System is a web application designed to allow users to access quizzes and attempt them online. The system provides features for managing and displaying quizzes, organizing quiz categories, and handling user authentication and quiz attempts, wherever these features are implemented.

The purpose of this test plan is to describe how the application will be tested to verify that its features work correctly and meet the project requirements.

2. Objectives of Testing

The main objectives of testing are:

- To verify that the application works according to its requirements.
- To check whether users can register and log in successfully.
- To verify that quizzes and categories are displayed correctly.
- To check whether users can view and attempt quizzes.
- To verify that quiz responses and results are handled correctly.
- To identify errors and unexpected behaviour.
- To check communication between the frontend, backend, and database.
- To ensure that invalid inputs are handled properly.

3. Scope of Testing

3.1 Features Included

The following features will be tested, depending on their implementation in the application:

1. **User Authentication:** Registration, login, logout, and invalid login attempts.
2. **Quiz Catalogue:** Displaying available quizzes and organizing quizzes by category.
3. **Quiz Display:** Opening a quiz and displaying its questions and answer options.

4. **Quiz Attempts:** Selecting answers, submitting responses, and displaying results if supported.
5. **Category Management:** Creating, viewing, updating, or deleting categories if these operations are available.
6. **Backend API:** Checking API requests, responses, and error handling.
7. **Database Operations:** Verifying that user, quiz, category, and attempt data are stored and retrieved correctly.
8. **User Interface:** Checking navigation, buttons, layouts, and error messages.

3.2 Features Not Included

- Testing external services that are not part of the project.
- Large-scale performance testing under heavy production traffic.
- Formal security certification or professional penetration testing.

4. Testing Strategy

The following testing methods will be used.

4.1 Unit Testing

Individual functions and components will be tested separately to verify that they work correctly.

Example: Checking whether a function validates user input correctly.

4.2 Integration Testing

Integration testing will verify communication between different parts of the application.

Example: Checking whether quiz data retrieved by the backend is displayed correctly on the frontend.

4.3 System Testing

The complete application will be tested to verify that the main user workflows work as expected.

Example: Logging in, opening a quiz, answering questions, and submitting the quiz.

4.4 Functional Testing

Each feature will be checked against its expected behaviour.

Example: Verifying that clicking the View Quiz option opens the selected quiz.

4.5 Validation Testing

Valid and invalid inputs will be provided to check whether the application accepts correct data and handles incorrect data appropriately.

5. Test Environment

The following environment will be used for testing.

Component	Technology
Frontend	React.js
Backend	Node.js and Express.js
Database	MongoDB
Programming Language	JavaScript
Browser	Google Chrome
Code Editor	Visual Studio Code
Version Control	Git and GitHub
API Testing	Postman, if available
Operating System	Windows

The application will be tested locally and, where applicable, through its deployed version.

6. Test Cases

The following test cases cover the main workflows of the Online Quiz Management System.

Test ID	Test Scenario	Test Steps	Expected Result
TC01	User registration	Enter valid registration details and submit.	The user should be registered successfully if registration is supported.
TC02	Valid login	Enter valid credentials and log in.	The user should be authenticated successfully.
TC03	Invalid login	Enter incorrect credentials.	An appropriate error message should be displayed.

TC04	Empty login fields	Submit the login form without required fields.	Required-field validation should be displayed.
TC05	View quiz catalogue	Open the quiz catalogue page.	Available quizzes should be displayed correctly.
TC06	View quiz details	Select a quiz and click View Quiz.	The selected quiz should open with its relevant details.
TC07	Display quiz questions	Open a quiz containing questions.	Questions and answer options should be displayed correctly.
TC08	Select an answer	Choose an answer for a question.	The selected answer should be recorded in the interface.
TC09	Submit a quiz	Answer the questions and submit the quiz.	The answers should be submitted and the expected result should be displayed if supported.
TC10	Open a quiz category	Select a category.	The relevant quizzes should be displayed if category filtering is implemented.
TC11	Empty quiz catalogue	Test with no quizzes available.	A suitable empty-state message should be displayed.
TC12	Invalid API request	Send an invalid request to a backend API.	The API should return an appropriate error response.
TC13	Database retrieval	Request existing quiz data.	The correct quiz data should be retrieved from the database.
TC14	Page navigation	Navigate between available pages.	The correct page should open without unexpected errors.
TC15	Responsive interface	Open the application on different screen sizes.	The interface should remain usable and readable.

Note: Test cases for features not yet implemented should be marked Not Applicable or Pending, rather than Pass.

7. Entry Criteria

Testing can begin when:

- The required application code is available.

- The frontend and backend can be started.
- The database connection is configured.
- The required test data is available.
- The features selected for testing are ready.

8. Exit Criteria

Testing can be considered complete when:

- All applicable planned test cases have been executed.
- Critical defects have been resolved or documented.
- Test results have been recorded.
- Remaining issues have been reported to the development team.
- The application meets the agreed acceptance requirements.

9. Defect Reporting

When a defect is identified, it should be recorded with the following details:

- Defect ID
- Defect title
- Description of the problem
- Steps to reproduce the problem
- Expected result
- Actual result
- Severity
- Status

Defect Severity

Severity	Description
Critical	A major failure that prevents the application from functioning.
High	An important feature does not work correctly.
Medium	A feature works incorrectly in some situations.
Low	A minor interface or usability issue.

10. Risks and Mitigation

Risk	Mitigation
Database connection failure	Verify database configuration and connection status.
Invalid user input	Test validation with valid and invalid inputs.
API communication errors	Test backend endpoints and inspect error responses.
Missing quiz data	Prepare test data before testing.
Browser compatibility issues	Test the application in supported browsers.
New code introduces defects	Perform regression testing after changes.

11. Responsibilities

- **Development Team:** Implement features and fix identified defects.
- **Testing Team:** Execute test cases, record results, and report defects.
- **Project Team:** Review the test results and decide whether the application meets the project requirements.

For a student project, the same team members may perform both development and testing.

12. Test Deliverables

The following deliverables will be maintained:

1. Software Test Plan document.
2. Test case list.
3. Test execution results.
4. Defect report.
5. Final testing summary.

13. Conclusion

The Software Test Plan provides a structured approach to testing the Online Quiz Management System. By testing the individual components, their integration, and the complete application, the team can identify defects and verify that the implemented features work as expected.

Testing will help improve the reliability, usability, and correctness of the application before its final submission.